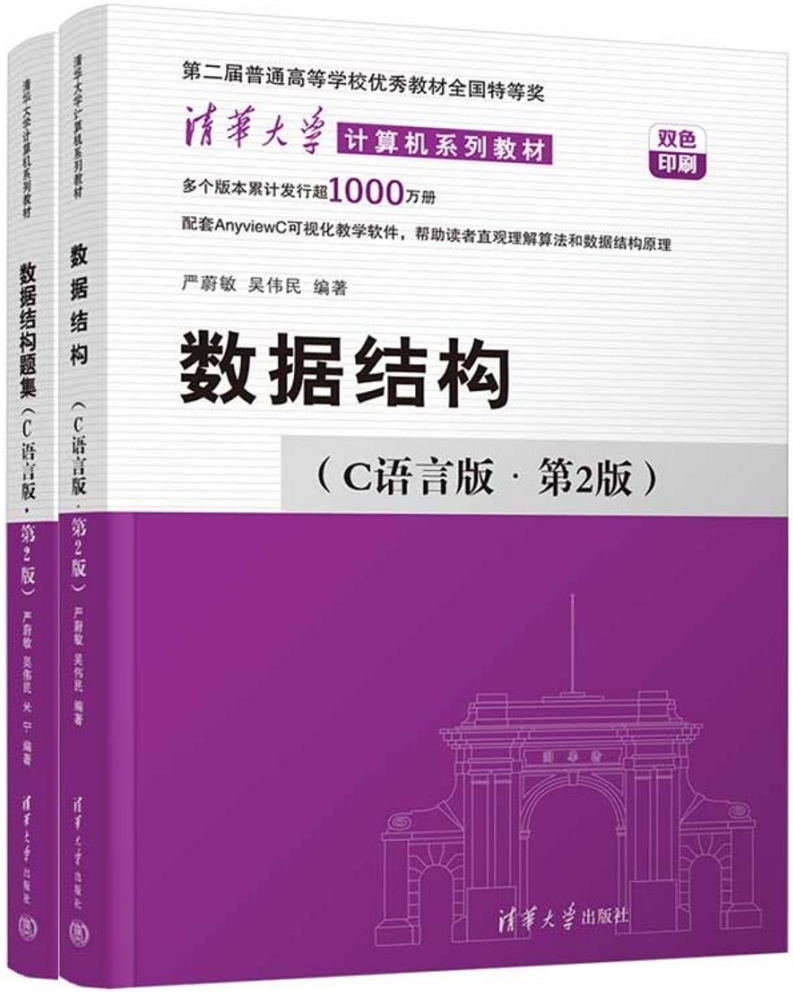


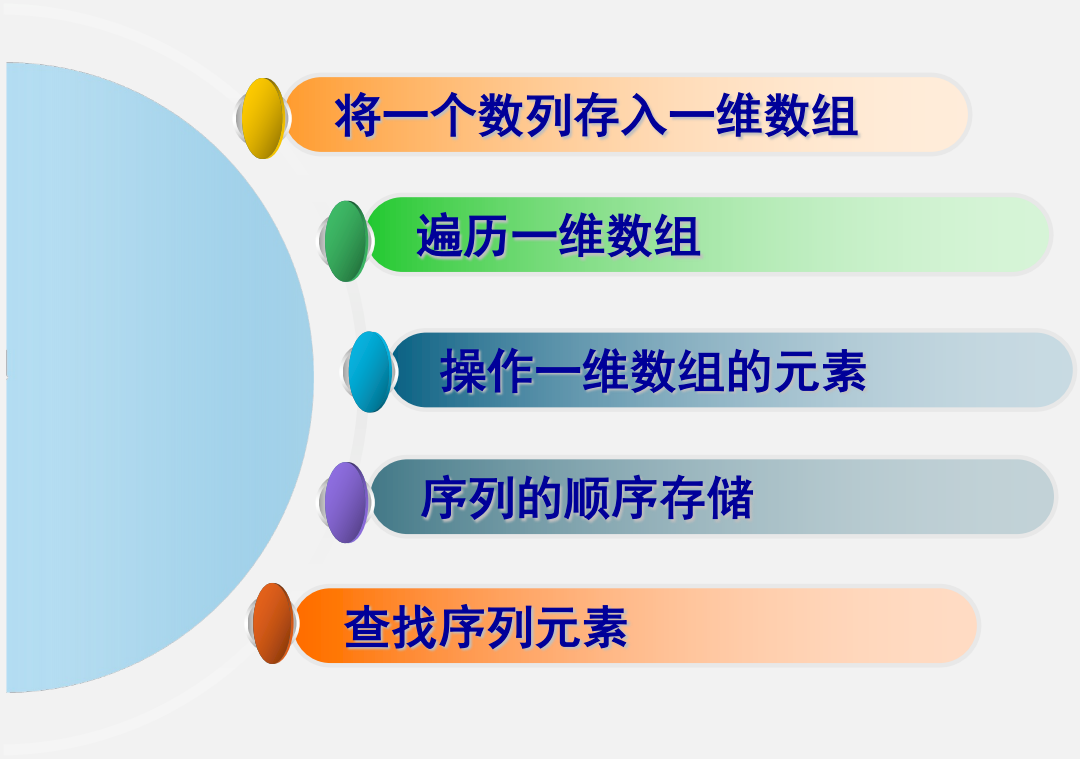
清華大学出版社
TSINGHUA UNIVERSITY PRESS

第1章

绪 论

1.0 一维数组与序列（补充内容：基础复习与增强）

清華大學出版社
TSINGHUA UNIVERSITY PRESS





数组和序列

清華大學出版社
TSINGHUA UNIVERSITY PRESS

数组是C语言提供的最常用的结构类型之一。假如需要处理N个数值，可以如下声明一个一维数组v：

```
ElemType v[N];
```

其中，ElemType是数值的类型，N是数值个数的常量。既可将每个数值分别保存在不同的变量（数组v的元素，也称为下标变量）中，也便于对这N个数值进行批量处理，包括求最大值、最小值和平均值，等等。



1.0.1 将一个数列存入一维数组

清華大學出版社
TSINGHUA UNIVERSITY PRESS

将一个数列（或序列）存入一维数组有多种方式。

1.0.1.1 在声明数组时初始化

1. `int v[10]={1,2,3,4,5,6,7,8,9,10};`
2. `int v[]={1,2,3,4,5,6,7,8,9,10};`
3. `int v[10]={1,2,3};`
4. `int v[10];`

在v的上述4个不同声明中，v的长度均指定为10。

☆ 第1、2个声明均对v的10个元素依次初始化为整数1到10；

☆ 第3个只将v前3个元素依次赋初值1、2和3，其余元素为0；

☆ 第4个未对v的元素赋初值。多数C编译器产生的代码都令变量取缺省值，如整型变量初值为0。



1.0.1 将一个数列存入一维数组

清華大學出版社
TSINGHUA UNIVERSITY PRESS

1.0.1.2 使用循环对数组元素依次赋值

当声明的数组很大或者数组元素与其下标存在对应关系时，可以使用循环赋值。

```
int v[60];
for(int i=0; i<60; i++)
  v[i]=i;
```

也可以将一个数组的值赋给另一个数组。

```
char a[7],
     b[7]={'p','r','o','g','r','a','m'};
for(int i=0; i<7; i++)
  a[i]=b[i];
```

1.0.1 将一个数列存入一维数组

1.0.1.3 数组作为参数

C语言的函数可用整个数组作为参数。通常（尤其是分配空间和使用空间不相同时）不写数组参数的长度，只写一对空的方括号“[]”。例如，

```
float average(float a[], int n) {
    float sum = 0;
    for(int i=0; i<n; i++)
        sum += a[i];
    return sum/n;
}
```

1.0.2 遍历一维数组

清華大學出版社
TSINGHUA UNIVERSITY PRESS

所谓遍历一维数组就是对数组的每个元素访问1次且仅1次。最自然简捷的方法是使用循环，利用下标或数组元素类型的指针，依次访问数组a的元素。

```
for(int i=0; i<7; i++)
    printf("%c",a[i]);
```

或者

```
for(int i=0; i<7; i++)
    printf("%c",*(a+i));
```

或者

```
char *p,*q;
for(q=a+7,p=a; p<q; p++)
    printf("%c",*p);
```

1.0.3 操作一维数组的元素

清華大學出版社 TSINGHUA UNIVERSITY PRESS

设有声明，并对一维数组d的所有元素赋以随机数

```
int d[10];
for(int i=0; i<10; i++)
  d[i]=random(100);
```

1. 对d的元素求和（连加）

```
int sum=0;
for(int i=0; i<10; i++)
  sum+=d[i];
```

2. 对d的元素求积（连乘）

```
int product=1;
for(int i=0; i<10; i++)
  product*=d[i];
```

3. 对d中下标为偶数的元素求和

```
int sum2=0;
for(int i=0; i<10; i+=2)
  sum2+=d[i];
```

4. 对d中下标为奇数的元素求和

```
int sum1=0;
for(int i=1; i<10; i+=2)
  sum1+=d[i];
```

5. 对d中每个元素求加上x

```
for(int i=0; i<10; i++)
  d[i]+=x;
```

6. 对d中每个元素求减去x

```
for(int i=0; i<10; i++)
  d[i]-=x;
```

1.0.3 操作一维数组元素

清華大學出版社
TSINGHUA UNIVERSITY PRESS

7. 对d中每个元素求乘上x

```
for(int i=0; i<10; i++)
    d[i]*=x;
```

8. 对d中每个元素求除以x

```
for(int i=0; i<10; i++)
    d[i]/=x;
```

9. 求d中元素的最大值max

```
int max=d[0]
for(int i=1; i<10; i++)
    if(d[i]>max)max=d[i];
```

10. 求d中元素的最小值min

```
int min=d[0]
for(int i=1; i<10; i++)
    if(d[i]<min)min=d[i];
```



1.0.4 序列的顺序存储结构

清華大學出版社
TSINGHUA UNIVERSITY PRESS

可以用一维数组存储序列。但在许多情况下，需要作一些变通，才便于对序列的存储和各种操作。

1.0.4.1 数组及其长度的封装

当程序中有多个长度不尽相同的数组时，最好将每个数组与其长度封装为一个结构。

```
struct {
    ElemType  elem[N];
    int  length=N;
} d;
```

1.0.4 序列的顺序存储结构

清華大學出版社 TSINGHUA UNIVERSITY PRESS

1.0.4.2 长度待定序列的存储结构

当序列的长度待定时，可将其元素的存储空间动态分配，并与其长度封装为一个记录。右边是一个序列的典型存储结构的类型定义：

```
typedef struct {
     ElemType  *elem;
     int length;
  } *Sequence;
```

声明一个序列变量时，启用前需要对其分配存储空间。

```
Sequence InitSequence(int n) {
  Sequence s;
  if (!(s=(Sequence)malloc(sizeof(*s)))) exit(OVERFLOW);
  if (!( s->elem=(ElemType*)malloc(n*sizeof(ElemType))))
     exit(OVERFLOW);
  s->length=n;
  return s;
}
```

1.0.5 顺序查找序列元素

清華大學出版社 TSINGHUA UNIVERSITY PRESS

在序列中查找元素的基本方法是顺序查找，平均时间复杂度为O(n)。

```
int SeqSearch(Sequence s, ElemType x) {
  int i;
  for (i=0;
       i<s->length && notEqual(x,s->elem[i]);
       i++);
  if (i<s->length) return i;
  return -1;
}
```

查找是最常用的操作，若在无序序列查找，则常用顺序查找（Sequential Search）。基本思想：从待查序列的一端开始顺序扫描，依次将元素和给定值比较。若当前元素与给定值相等，则查找成功；反之，若扫描结束后，仍未找到，则查找失败。

1.0.6 折半查找有序序列元素

清華大學出版社 TSINGHUA UNIVERSITY PRESS

在有序序列中可采用折半查找的方法查找元素，平均时间复杂度下降为$O(\log n)$。

```
int binSearch(Sequence s, ElemType x) {
 int low,mid,high;
 low=0;  high=s->length-1;
 while(low<=high) {
    mid=(low+high)/2;
    if(Equal(x,s->elem[mid]))return mid;
    else if(less(x,s->elem[mid]))high=mid-1;
    else low=mid+1;
 }
 return -1;
}
```

1.0.7 序列的元素滑移（换位）

清華大學出版社
TSINGHUA UNIVERSITY PRESS

1.0.7.1 两个元素交换位置

```
int swap(Sequence s, int i, int j) {
    ElemType t;
    if (i<0 || i>=s->length || j<0||j>=s->length)
        return 0;
    if (i!=j) {
        t=s->elem[i];
        s->elem[i]=s->elem[j];
        s->elem[j]=t;
    }
    return 1;
}
```

1.0.7 序列的元素滑移（换位）

清華大學出版社
TSINGHUA UNIVERSITY PRESS

1.0.7.2 1个元素向前和向后滑移m个位置

```
int moveForward1(Sequence s, int i, int m) {
    ElemType t;
    int j;
    if (i<0 || i>=s->length || m<0 || m>i)
      return 0;
    if (m!=0) {
      t=s->elem[i];
      for (j=i; m>0; j--,m--)
        s->elem[j]=s->elem[j-1];
      s->elem[j]=t;
    }
    return 1;
}
```

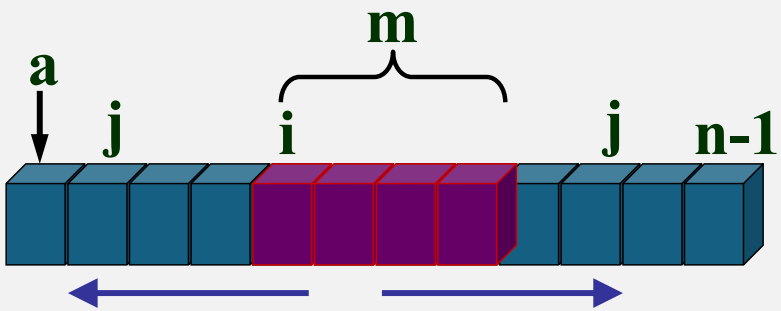
1.0.7 序列的元素滑移（换位）

清華大學出版社
TSINGHUA UNIVERSITY PRESS

1.0.7.3 连续子序列的前移和后移m个位置

例 移动子数组。

一维数组——




```
int move(char *a, int n, int i, int m, int j)
```

1.0.7 序列的元素滑移（换位）

1.0.7.3 连续子序列向前和向后滑移m个位置

例如，序列s中从第i元素起n个字符向后滑移m个位置（实际上，与后面的子序列换位）：

```
int moveBackwardx(Sequence s, int i, int n, int m) {
  ElemType *t;
  int j,k;
  if (i<0 || n<0 || i+n-1>=s->length || m<0
      || m>S->length-(i+n))
    return 0;
  if (m!=0 && n!=0) {
    t=(ElemType*)malloc(n*sizeof(ElemType));
    if(t==NULL) return 0;
    for (j=0; j<n; j++) t[j]=s->elem[i+j];      // 暂存要滑移的子序列
    for (j=i; m>0; j++,m--) s->elem[j]=s->elem[j+n]; //之后子序列前移
    for (k=0; k<n; j++,k++) s->elem[j]=t[k];    // 滑移的子序列到位
  }
  return 1;
}
```

1.0.8 关于序列的元素插入和删除

清華大學出版社
TSINGHUA UNIVERSITY PRESS

如果在序列中插入或者删除元素，其顺序存储结构中部分元素需要移动，平均时间复杂度为$O(n)$。

显然，顺序存储结构只适合静态序列。对于动态序列，需要寻找时间复杂度为$O(1)$的插入和删除操作的实现方法。

如何存储集合？

如何实现集合的各种运算？

从专业学习和发展阶段看：

学习、研究**数据结构和算法的设计、实现与分析**，势在必行！

1.1 什么是数据结构?

1.1.1 抽象与结构（增补内容）

抽象（abstract）是对一类事物或一个系统的简化描述。它舍弃个别的、非本质的属性，抽取共同的、本质的属性，是形成概念的必要手段。

在计算机学科的三种形态中：**抽象**是连接**理论**和**设计**的**纽带**。

例 用几秒钟就能记住整数集合：

{ 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30 }

但是，如果要用同样时间记住：

{ 18, 2, 20, 16, 24, 4, 30, 26, 6, 22, 14, 12, 28, 10, 8 }

就难多了！

前一个集合是有序的，容易发现和记忆它的一个有限**结构**：

2 和 30 之间的偶数集。

后一个集合没有这样的**结构**，因此不易记住。

抽象和结构——克服复杂性的重要认知形式

在认识事物时，对结构的**分解**和**合成**十分重要。

例如，一辆汽车能够描述为“**由人操纵的、燃油动力的和陆上行驶的运输设备**”。

方向盘由人操纵，引擎通常由燃油获得动力，加上车轮和车厢后构成了运输的功能。

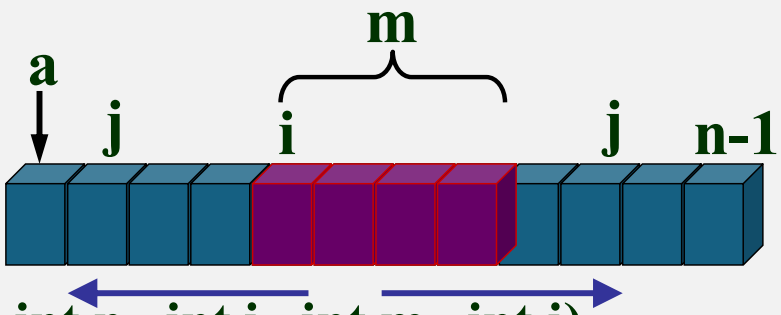
但是每一部分都可以放到别的机器上，没有哪部分能够单独被称为汽车。构成汽车的是所有部件的正确**组装**（composition），或者说**协同作用**（synergy）。

20

功能抽象

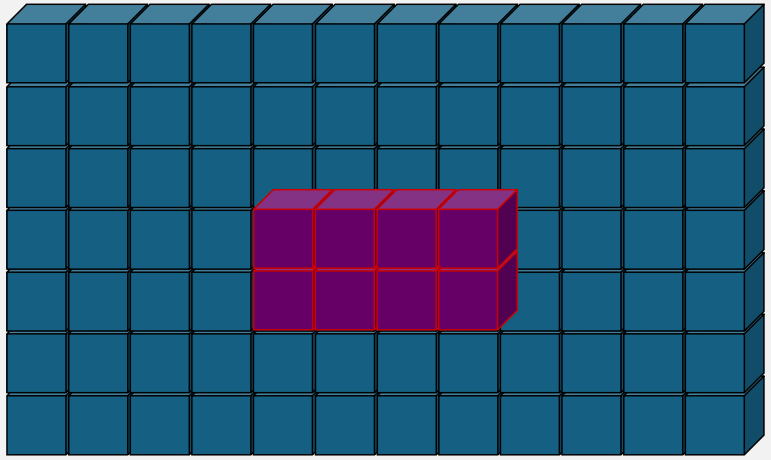
例 移动子数组。

◆一维数组——




```
int  move(char *a,  int n,  int i,  int m,  int j)
```

◆ 二维数组——



1.1 什么是数据结构?

1.1.1 数据结构讨论的范畴

你同意吗?

计算机70~90%运行时间是进行查找、排序;

科学家20~50%工作时间是进行资料检索、分析;

毕业生20~70%日常时间是进行求职（找饭碗）;

……

你每天花大量的时间在进行各种查找（物品、知识）;

结论:

生活和工作中，人们进行种种寻觅！（主动或被动）

其中，有些是在**既定范围**查找，有些则是**未知范围**!

计算机如何高效地查找、排序?

清華大學出版社
TSINGHUA UNIVERSITY PRESS

What 找什么? 计算机处理的对象是什么?

Where 在哪里?

How 如何找?

计算机如何表示和处理形形色色的各种对象?

如何存储要处理的对象?

如何增加、删除和修改对象?

如何在所存储的对象集合中找到当前需要处理的对象?

……

计算机科学是一门研究用计算机进行**信息表示和处理**的学科。这涉及两个问题：

信息的表示

信息的处理

直接影响处理信息的程序的**效率**。随着计算机普及，信息量暴增，信息范围拓宽，使许多系统程序和应用程序的规模巨大，且结构相当复杂。

因此，为了编写出一个“**好**”的程序，必须分析待处理的对象的特征及各对象之间存在的关系，这就是数据结构这门课程所要研究的主要问题。

Pascal之父Niklaus Wirth:

Algorithms + Data Structures = Programs

程序设计：为计算机处理问题编制指令序列

算法：处理问题的策略

数据结构：问题的数学模型

非数值计算的程序设计问题

例1-0 求一组(n个)整数中的最大值

算法？ 基本操作是“比较两个数的大小”

模型？ 取决于整数值的范围和组织形式

如：序列、有序序列、集合、树 等

例1-1 **目录、索引、字典、查询系统和浏览器。**本书的前后分别配有目录和索引，都是为了方便读者查阅书中内容而设的查找工具。

在计算机实现各种查询或查阅系统，用到了数据结构中的各种查找表及其算法。人们也将这些查找表形象地泛称为“字典”，或者视为各种用途的字典的不同表示和实现。

例1-2 计算机和人对弈问题。

算法 ？ **对弈的规则和策略**

模型 ？ **棋盘及棋盘的格局**

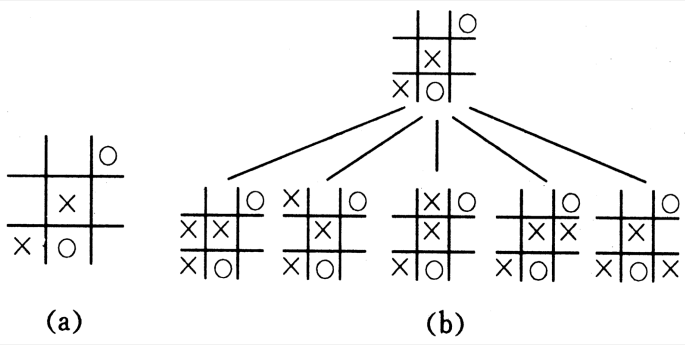


图1.2 井字棋对弈“树”
(a)棋盘格局示例；(b)对弈树的局部

井字棋由两人对弈，棋盘为3×3的方格。
当一方的3个棋子占同一行，或同一列，
或同一对角线时便为胜方。

若将从对弈开始到结束的过程中所有可能出现的格局都画在一张图上，则可得到一棵倒长的“**树**”。“**树根**”是对弈开始之前的棋盘格局，而所有的“**叶子**”就是可能出现的结局。对弈的过程就是从树根沿树权到某个叶子的过程。“树”可以是某些非数值计算问题的数学模型，它也是一种数据结构。

例1.3 多岔路口交通管理

图1.3(a) 是一个五岔路口，
其中，**C** 和 **E** 为单行道，
共有 **13** 条可通行路线。
有的**不可**同时通行，如 **E** → **B** 和 **A**→**D**。
但有的**可以**同时通行，如 **A** → **B** 和 **E**→**C**;
如何设置交通灯进行该路口车辆通行管理呢**?**

这类交通、道路问题的数学模型是一种称为“**图**”的**数据结构**。
可用 **1** 个**顶点**（圆圈）表示 1 条通行路线，
冲突，不能同时通行的线路的**顶点**用**线 (边)** 相**连接**，
如线路 **AD** 与 **EC** 冲突，它们之间用线相连接;
AD 与 **EA** 也冲突，之间也连线;
显然， **EA** 与 **AB** 也是冲突的，连线。
可以自行验证：所有冲突线路都可这样连线。
为什么会有 3 个孤立顶点（线路）？——都是右转线路!

如此表示线路及其冲突关系的数据结构“**图**”，如何解决交通灯进配置问题呢?
这等价于**图的顶点着色问题**:
(1) 每个顶点染 **1 种颜色**;
将对应路口中心交通灯颜色
(2) 有线相连的顶点染**不同色**;
灯色让不冲突线路同时通行
(3) 尽可能用**最少颜色**。
使得灯色切换最少，追求效率

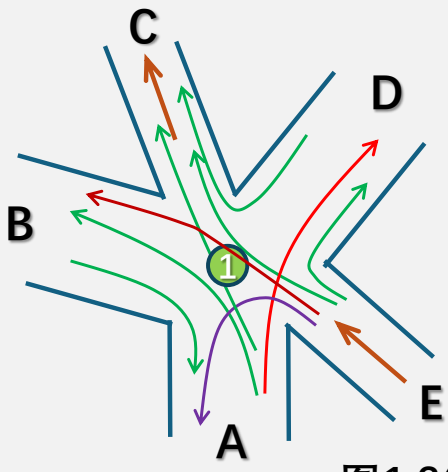
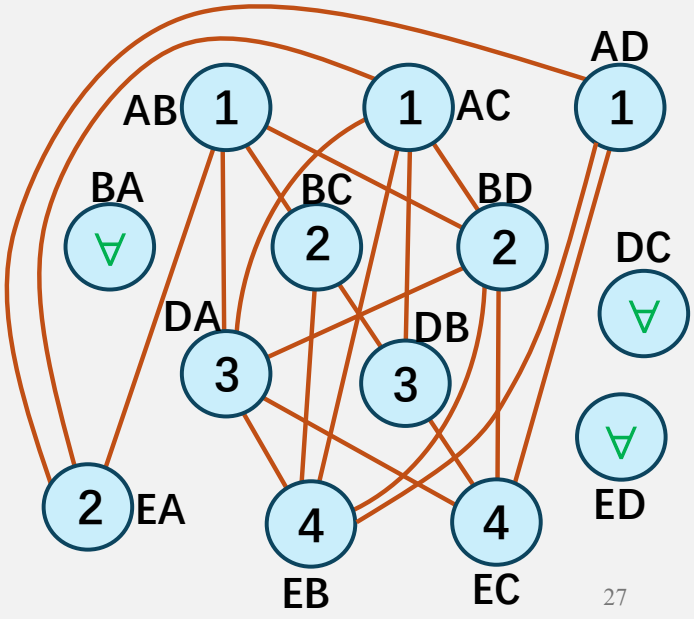


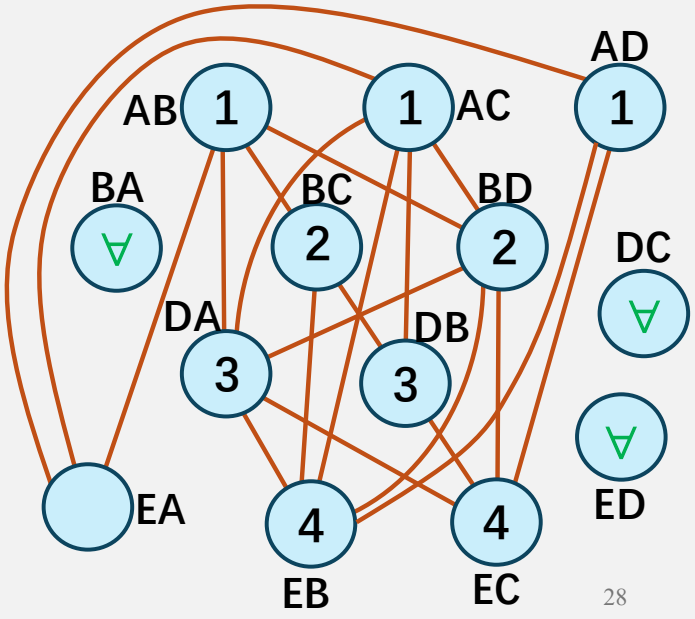
图1.3(a)



五叉路口交通线路通行分组操作：

1. 三个孤立顶点（右转线路）不受灯控限制，确认后移除
2. AB、AC、AD3个顶点染颜色1，被移除及其相关边
3. 现在EA成为孤立点，即它除了颜色1组外，与其他线路不冲突，在其他灯色时可通行，移除；
4. 余下6个顶点可两两一组，划分为3组，对应颜色2、3和4。

交通灯颜色及对应可通行线路如下图：



从这3个例子可见，描述这类非数值计算问题的数学模型不再是数学方程，而是诸如表、树和图之类的数据结构。

由此可概括地说，**数据结构**是讨论“现实世界各种事物的**数学模型**（非数值计算）以及其上的**操作**在计算机中如何**表示**和**实现**”的学科分支。

也可以表述为：**数据结构**是一门研究非数值计算的程序设计问题中计算机的处理对象及其之间的关系和操作等的学科。具体实现的各种数据结构是对程序设计语言描述和操纵数据功能的扩展。

“数据结构”是介于数学、计算机硬件和计算机软件三者之间的一门核心课程（如图1.4所示）。在计算机科学中，“数据结构”不仅是一般程序设计（特别是非数值计算的程序设计）的基础，而且是设计和实现编译程序、操作系统、数据库系统、大数据模型及其他系统程序和大型应用程序的重要基础。

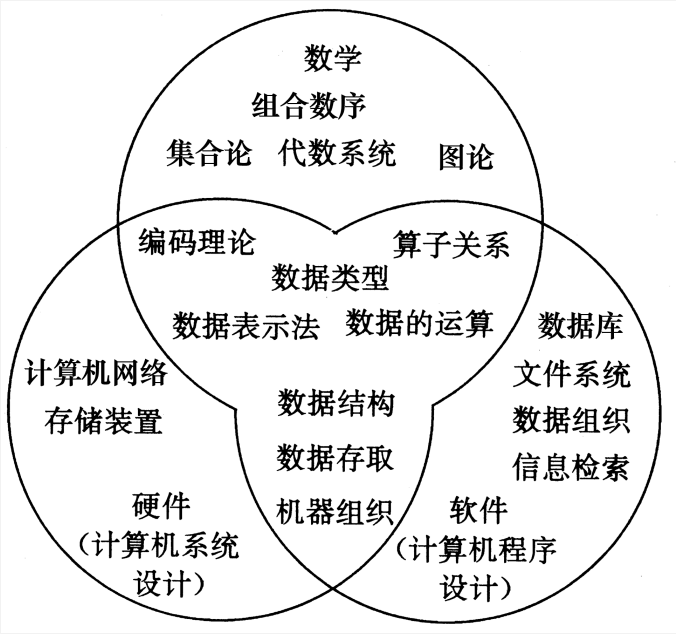


图1.4 “数据结构”所处的地位

1.2 基本概念和术语

一. 数据与数据结构

数据：

- 所有能**被输入**到计算机中，且被计算机**处理的符号**的**集合**。
- 是**计算机操作的对象**的总称。
- 是计算机处理的**信息**的某种特定的符号**表示形式**。

数据元素：

- **数据**（集合）中的一个**“个体”**
- **数据结构**中讨论的**基本单位**

数据项：数据结构中讨论的**最小单位**

数据元素是数据项的集合

例如：运动员（数据元素）

姓名	俱乐部名称	出生日期			加入日期			位置	业绩
		年	月	日	年	月	日		
		组合项1			组合项2				

- **关键字**（Key），常简称**键**，是数据元素（或记录）的关联值或其中某个数据项的值，用它可以标识（识别）一个数据元素（或记录）。
- 若此关键字可以唯一地标识一个记录，则称此关键字为**主关键字**(Primary Key)（对不同的记录，其主关键字均不同）。
- 反之，称用以识别若干记录的关键字为**次关键字**（Secondary Key)。
- 当数据元素只有一个数据项时，其关键字即为该数据元素的**值**。
- 除了主关键字之外的其他内容称为**值**，一个记录也就被称为**键-值对**。

数据对象（**data object**）是性质相同的数据元素的集合，是数据的一个子集。

例如，整数数据对象是集合N={0，±1，±2,…}，字母字符数据对象是集合C={'A','B',…,'Z')。

数据结构： 是相互之间存在一种或多种特定关系的数据元素的集合。

亦即带**结构**的数据元素的集合

数据的逻辑结构

形式上，数据结构是一个二元组

Data_Structures = (D, S)

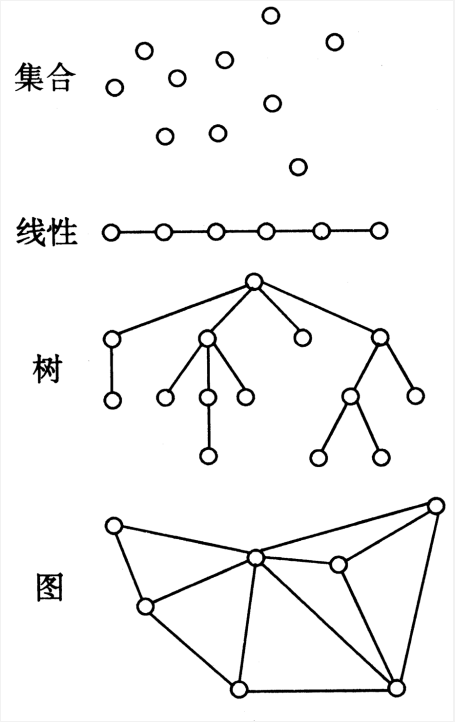
其中：D是数据元素的有限集，

S是D上关系的有限集。

上述数据结构的定义仅是对操作对象的一种数学描述，换句话说，是从操作对象抽象出来的数学模型。

结构定义中的“关系”描述的是数据元素之间的逻辑关系，因此又称为数据的逻辑结构。

数据的逻辑结构可归结为四类：



数据的存储结构

清華大學出版社 TSINGHUA UNIVERSITY PRESS

- 数据结构在计算机中的表示（又称**映像**）称为数据的**物理结构**，又称**存储结构**，它包括数据元素的表示和关系的表示。
- 表示信息的最小单位是二进制数的**一位**，叫做**位**（bit)。
- 由若干位组合形成的**位串**表示一个数据元素（如用一个字长的位串表示一个整数，用8位二进制数表示一个字符等），通常称这个位串为**元素**①（element）或**结点**（node)。
- 当数据元素由若干数据项组成时，位串中对应于各个数据项的子位串称为**数据域**（data field)。因此，元素或结点可看成是数据元素在计算机中的**映像**。

逻辑结构在存储器中的映象

数据元素的映象方法：

用二进制位(bit)的位串表示数据元素

例如：

$(321)_{10} = (501)_8 = (101000001)_2$

$A = (101)_8 = (001000001)_2$

关系的映象方法（表示<x, y>的方法）

顺序映象

以相对的存储位置表示后继关系。

例如:令 y 的存储位置和 x 的存储位置之间差一个常量C,而C是一个隐含值，整个存储结构中只含数据元素本身的信息。

如图1.6(a)为表示复数z1=3.0-2.3i和z2=-0.7+4.8i的顺序存储结构。

链式映象

以附加信息(指针)表示后继关系〈x,y〉需要用一个和 x 在一起的附加信息指示 y 的存储位置。

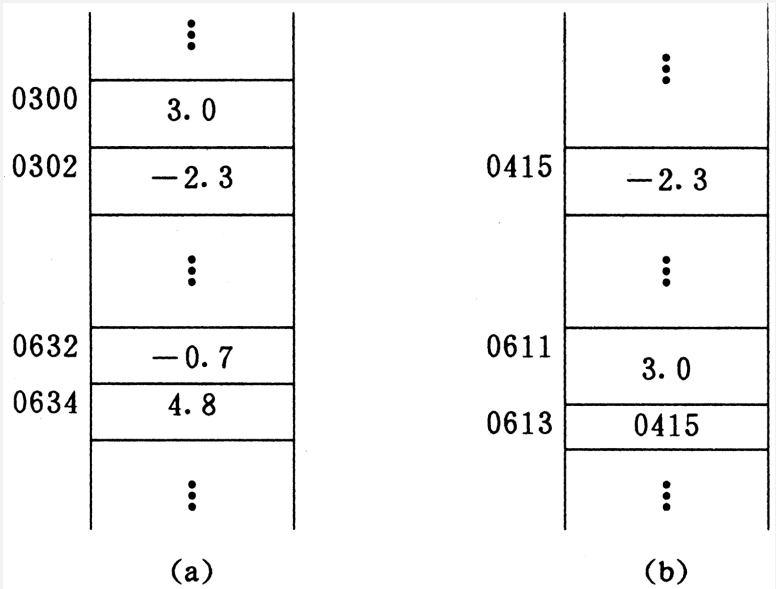


图1.6　复数存储结构示意图
(a)顺序存储结构；(b)链式存储结构

清華大學出版社 TSINGHUA UNIVERSITY PRESS

在不同的编程环境中，存储结构可有不同的描述方法。

用高级程序设计语言编程，可用语言中提供的**数据类型**描述之。

二. 数据类型

在用高级程序语言编写的程序中，必须对每个变量、常量或表达式，明确说明它们所属的**数据类型**。

数据类型是一个值的集合和定义在此集合上的一组操作的总称。

例如，C语言中 int 类型的整型变量

- 值集为某个区间上的整数（区间大小依赖于不同的机器），
- 定义在其上的操作为加、减、乘、除和取模等算术运算。

又如，以三个带有次序关系的整数表示一个长整数时，可利用C语言中提供的整数数组类型，定义长整数为：

```
typedef int Long_int [3];
```

清華大學出版社 TSINGHUA UNIVERSITY PRESS

三. 抽象数据类型（Abstract Data Type 简称ADT）

是指一个**数学模型**以及定义在此数学模型上的一组**操作**

ADT有两个重要特征:

数据抽象

用ADT描述程序处理的实体时，强调的是其**本质的特征、其所能完成的功能**以及它和**外部用户的接口**（即**外界使用它的方法**）

数据封装

将实体的**外部特性和其内部实现细节分离**，并且**对外部用户隐藏其内部实现细节**

抽象数据类型的描述方法

抽象数据类型可用（D，S，P）三元组表示，其中，D 是数据对象；

S 是D上的关系集；

P 是对D的基本操作集。

抽象数据类型的定义仅取决于它的一组逻辑特性，而与其在计算机内部如何表示和实现无关，即不论其内部结构如何变化，只要它的数学特性不变，都不影响其外部的使用。

一个含抽象数据类型的软件模块通常应包含定义、表示和实现3个部分。

按值的不同特性，抽象数据类型可细分为下列3种类型：

原子类型（atomic data type） 属原子类型的变量的值是不可分解的。这类抽象数据类型较少，因为一般情况下，已有的固有数据类型足以满足需求。但有时也有必要定义新的原子数据类型，例如数位为100的整数。

固定聚合类型（fixed-aggregate data type）属该类型的变量的值由确定数目的成分按某种结构组成。例如，复数是由两个实数依确定的次序关系构成。

可变聚合类型（variable-aggregate data type）和固定聚合类型相比较，构成可变聚合类型“值”的成分的数目不确定。

本书采用以下格式定义**抽象数据类型**：

ADT抽象数据类型名 {
  数据对象：<数据对象的定义>
  数据关系：<数据关系的定义>
  基本操作：<基本操作的定义>
}ADT抽象数据类型名

其中，数据对象和数据关系的定义用伪码描述，基本操作的定义格式为

基本操作名（参数表）
  初始条件：<初始条件描述>
  操作结果：<操作结果描述>

“**初始条件**”描述了操作执行之前数据结构和参数应满足的条件，若不满足，则放弃操作，或称操作失败，并返回相应出错信息。

“**操作结果**”说明了操作正常完成之后，数据结构的变化状况和应返回的结果。若初始条件为空，则省略之。

例1-6 抽象数据类型小整数集合的定义：

ADT SmallIntSet {

数据对象：**D={**i|i∈[1..n]**}** **// D的元素是区间[1..n]的整数，n是一个“不太大”的整数**

数据关系：**R={}** **// 元素之间无定义任何关系**

基本操作：

Init(S, n)
操作结果：构造并返回全集为{1,2,…,n}的空集S。
Free(S)
操作结果：集合S被回收，返回NULL。
Store(S, i)
初始条件：集合S已存在，1≤i≤n。
操作结果：将元素i添加至集合S中。若成功，则返回TRUE，否则返回FALSE。
Contain(S, i)
初始条件：集合S已存在，1≤i≤n。
操作结果：若集合中有元素i，则返回TRUE，否则返回FALSE。

Remove(S, i)
初始条件：集合S已存在，1≤i≤n。
操作结果：若成功移除集合中的元素i，返回TRUE。若无此元素，返回FALSE。
IsFull(S)
初始条件：集合S已存在。
操作结果：若集合已满(是全集)，返回TRUE；否则返回FALSE。
IsEmpty(S)
初始条件：集合S已存在。
操作结果：若集合S为空，返回TRUE；否则返回FALSE。
Size(S)
初始条件：集合S已存在。
操作结果：返回集合S中元素数量(集合的大小)。
}ADT SmallIntSet

1.3 抽象数据类型的表示与实现

清華大學出版社 TSINGHUA UNIVERSITY PRESS

抽象数据类型可通过固有数据类型来表示和实现，即利用处理器中已存在的数据类型来说明新的结构，用已经实现的操作来组合新的操作。

1.3.1 描述语言概要

本书在高级程序设计语言的虚拟层次上讨论抽象数据类型的表示和实现，并且讨论的数据结构及其算法主要是面向读者，采用C语言作为描述工具，有时也用伪码描述一些只含抽象操作的抽象算法。使得数据结构与算法的描述和讨论简明清晰，数据结构的类型定义和函数都是C语言代码，简要说明如下。

（1）预定义常量和类型

```
// 函数结果状态代码
#define   TRUE          1
#define   FALSE         0
#define   OK            1
#define   ERROR         0
#define   INFEASIBLE   -1
#define   OVERFLOW     -2
Typedef   int     Status;
    // 用作函数类型，其值是整数，
    // 作为函数结果状态代码
```

（2）数据结构的表示（存储结构）用类型定义（typedef）描述。数据元素类型约定为ElemType，由用户在使用该数据类型时自行定义。在讨论中，若无特别说明，能比较大小的ElemType类型为**int**或**char**等简单类型。

（3）基本操作定义为函数原型，实现为函数。当函数返回值为函数结果状态代码时，函数定义为**Status**类型。若需返回多个值时，则定义含这多个值的结构类型作为函数类型。

（4）基本函数

求最大值	max(表达式1,…,表达式n)
求最小值	min(表达式1,…,表达式n)
求绝对值	abs(表达式)
求不足整数值	floor(表达式)
求进位整数值	ceil(表达式)
判定文件结束	eof(文件变量)或eof
判定行结束	eoln(文件变量)或eoln

（5）逻辑运算约定

与运算**&&**：对于A**&&**B，当A的值为0时，不再对B求值。

或运算**||**：对于A**||**B，当A的值为非0时，不再对B求值。

（6）动态内存分配和回收函数

分配内存	**malloc**(单元数目*单元大小)
分配内存并清空	**calloc**(单元数目,单元大小)
重新分配内存	**realloc**(原空间指针,单元数目*单元大小)
释放内存	**free**(空间指针)

（7）算法紧凑格式

书中算法采用紧凑格式。类似保留字，内存管理函数名、花括号等也采用加粗体，以便阅读。

1.3.2 数据存储结构的C语言描述（复习与增强）

内存分区和动态分配

C语言程序的可访问内存分为4个区：

（1）栈，由编译器计算每个函数所需的内存块的大小，预分配参数和局部变量在块内的相对地址。在函数入口确定函数内存块在栈中的具体位置（基址），且在函数出口释放内存块。

（2）堆，在程序执行期间，可用库函数malloc、calloc和realloc在堆中申请动态分配所需内存，并在不用时用函数free释放所分配的内存。

（3）全局/静态区，是专门存储全局变量和静态变量的内存区。

（4）常量区，专门存放常量，常量只可取值，不可赋值。

C语言在栈中自动为函数分配和回收内存块，程序员无需也不能参与。但是，要熟悉函数的参数和局部变量在栈中存储机制，特别是参数的传递方式，这有助于深入理解函数嵌套调用和递归调用。

内存分区和动态分配

程序员能自由分配和使用的内存区域是堆。申请分配堆中内存块的库函数有：

```
void *malloc(unsigned int size);                    // 申请分配size个字节的内存块
void *calloc(unsigned int n, unsigned int size);    // 申请分配n*size个字节的内存块
```

参数size的实参一般用sizeof()求得，

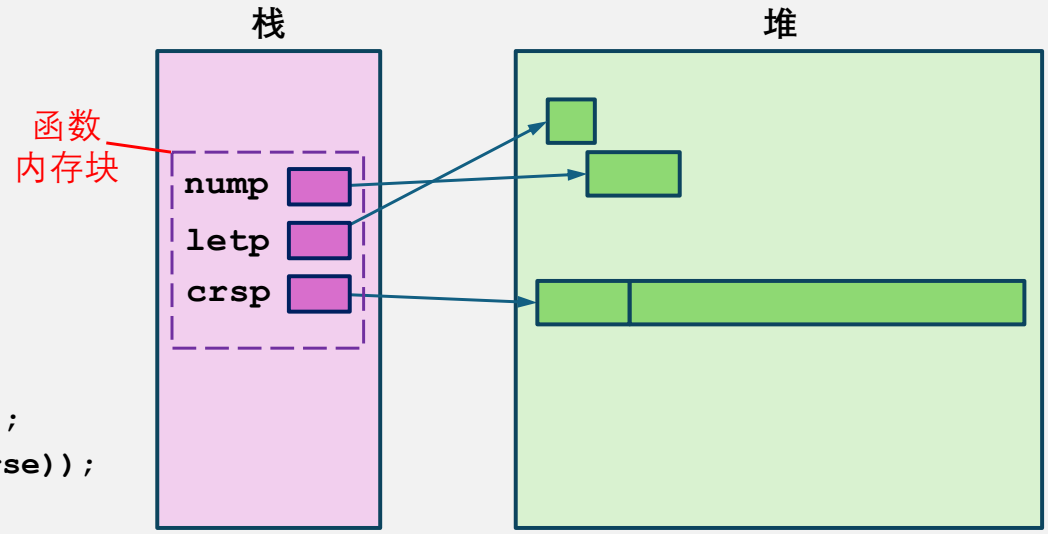
两个函数功能相近，函数值是指向分配所得的一块地址连续的内存块的起始地址（也称基址）的指针，若为空指针，则是申请失败。

calloc比malloc多一项功能，自动对所得的内存块按位清0，使其是一块“干净”的内存块。

例1 申请分配整数、字符和结构的存储空间

```
typedef struct {
  int  num;       // 课程编号
  char name[16]; // 课程名称
} Course;  // 课程结构类型
int    *nump;
char   *letp;
Course *crsp;
nump = (int*)calloc(1, sizeof(int));
letp = (char*)calloc(1, sizeof(char));
crsp = (Course*)calloc(1, sizeof(Course));
```

指针类型转换



例2 用引用参数返回函数内动态分配的存储空间

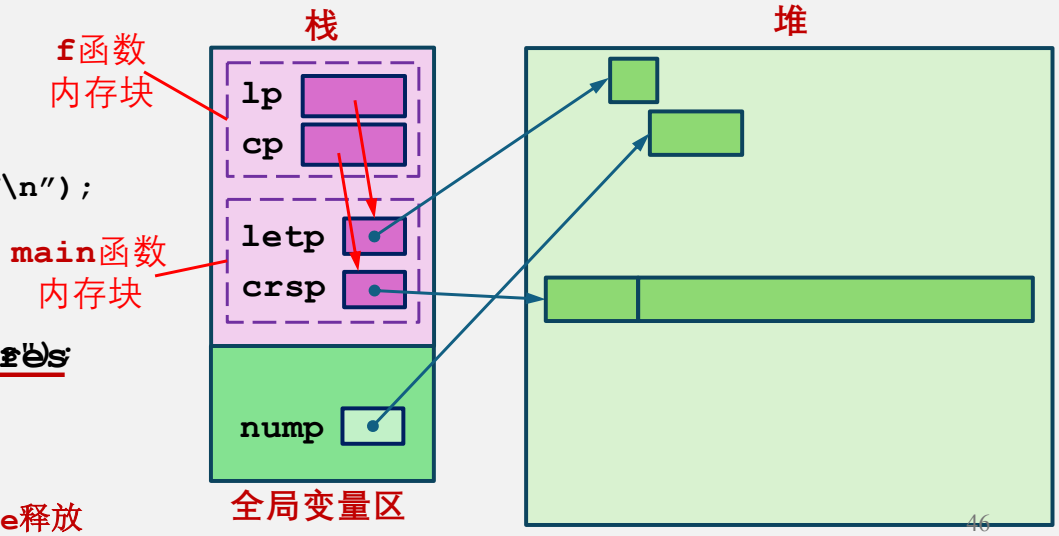
```
typedef struct {
  int  num;        // 课程编号
  char name[16]; // 课程名称
} Course;  // 课程结构类型
int *nump; // 全局变量
Status f(char* &lp, Course* &cp) { // 两个不同类型指针的引用形参
  if(!(nump = (int*)calloc(1, sizeof(int)))) return OVERFLOW;
  *nump = 69;
  if(!(lp = (char*)calloc(1, sizeof(char)))) return OVERFLOW;
  if(!(cp = (Course*)calloc(1, sizeof(Course)))) return OVERFLOW;
}
int main() {
  char   *letp;
  Course *crsp;
  if(!f(letp, crsp)) printf("OVERFLOW\n");
  else {
    *letp = 'A';
    crsp->num = 203;
    strcpy(crsp->name,"Data Structures");
  }
  ... free(letp);   free(nump);   ...
}
```

传地址

当分配的内存块不再使用时，可用库函数free释放

各种存储结构的元素存储空间基本采用在堆的动态分配
在函数中构建的存储结构，要用引用形参返回

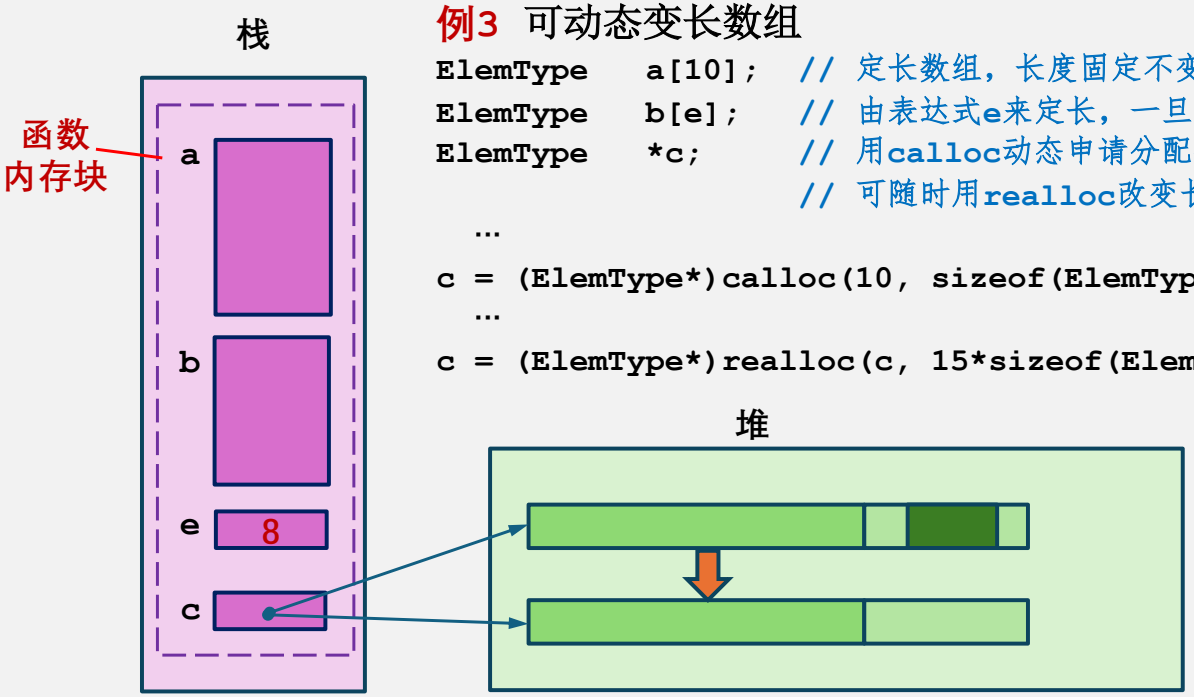
不再使用的存储空间要及时释放



当分配的内存块的太小或太大时，可用以下库函数申请改变内存块的大小：

```
void *realloc(void *p, unsigned int size);  // 申请将p所指内存块的大小改为size个字节
```

参数p是指向现有内存块的指针，函数返回指向改变大小后的内存块的指针，不一定与p相同。原内存块的内容迁移到改变后的内存块，若内存块是改小的，则有部分内容被截去。



例3 可动态变长数组

```
ElemType    a[10];   // 定长数组，长度固定不变
ElemType    b[e];    // 由表达式e来定长，一旦确定，长度固定不变
ElemType    *c;      // 用calloc动态申请分配指定长度的空间，
                     // 可随时用realloc改变长度
  …
c = (ElemType*)calloc(10, sizeof(ElemType)); // 动态生成长度为10的数组
  …
c = (ElemType*)realloc(c, 15*sizeof(ElemType)); // 改变c的长度为15
```

若后有足够的空闲空间，则延长

否则，分配并将内容转至新空间
回收原有空间

1.4 算法和算法分析

1.4.1 算法

算法（algorithm）是对特定问题求解步骤的一种描述，它是指令的有限序列，其中每一条指令表示一个或多个操作；此外，一个算法还具有下列5个重要特性：

（1）**有穷性** 一个算法必须总是（对任何合法的输入值）在执行有穷步之后结束，且每一步都可在有穷时间①内完成。

（2）**确定性** 算法中每一条指令必须有确切的含义，读者理时不会产生二义性。并且，在任何条件下，算法只有唯一的一条执行路径，即对于相同的输入只能得出相同的输出。

（3）**可行性** 一个算法是能行的，即算法中描述的操作都是可以通过已经实现的基本运算执行有限次来实现的。

（4）**输入** 一个算法有零个或多个输入，这些输入取自于某个特定对象的集合。

（5）**输出** 一个算法有一个或多个输出，这些输出是和输入有着某些特定关系的量。



1.4.2 算法设计的要求

通常设计一个“好”的算法应能达到以下目标

1. **正确性** **首先，**算法应当满足以特定的“规格说明”方式给出的需求。

   其次，对算法是否“**正确**”的理解可以有以下四个层次：

   a. 程序中不含语法错误；

   b. 程序对于几组输入数据能够得出满足要求的结果；

   c. 程序对于精心选择的、典型、苛刻且带有刁难性的几组输入数据能够得出满足要求的结果；

   d. 程序对于一切合法的输入数据都能得出满足要求的结果；

   通常以**第 c 层**意义的正确性作为衡量一个算法是否合格的标准。

2. **可读性** 算法首先应便于人的阅读与交流，其次才是实现为机器执行的代码。

3. **健壮性** 输入数据非法时，应能做出适当反应或处理，而不会产生莫明其妙的输出结果。

4. **高效率与低存储量需求** 执行时间短，所需存储空间少。



1.4.3 算法效率的度量

算法执行时间需通过依据该算法编制的程序，在计算机上运行时所消耗的时间来度量。而度量一个程序的执行时间通常有两种方法。

事后统计法 不同算法的程序可通过运行时间统计数据以分辨优劣。

缺点： 1. 必须执行程序

2. 其它因素掩盖算法本质

事前分析估算法

撇开与计算机硬件、软件有关的因素，可以认为一个特定算法“运行工作量”的大小，只依赖于问题的规模（通常用整数量n表示），或者说，它是问题规模的函数。

清華大學出版社
TSINGHUA UNIVERSITY PRESS

一般情况下，算法中基本操作重复执行的次数是问题规模n的某个函数$f(n)$，算法的时间量度用**大O表示法**记作

$$T(n) = O(f(n)) \tag{1-5}$$

存在常数$c>0$和整数$n_0>0$，使得对所有$n \geq n_0$，满足$T(n) \leq cf(n)$。它表示$f(n)$是$T(n)$的**渐进上界**，随问题规模n的增大，算法执行时间的增长率和$f(n)$的增长率相同，称为算法的**渐近时间复杂度**（asymptotic time complexity），简称**时间复杂度**。

例如，$T(n)=2n^3+80n^2+15$是$O(n^3)$的。设$c=3$，当$n \geq 80$时，$T(n) \leq cn^3$。如图1.8所示，$n_0=80$是一个临界点，随着n的继续增大，$T(n)$曲线都在$3n^3$曲线之下。

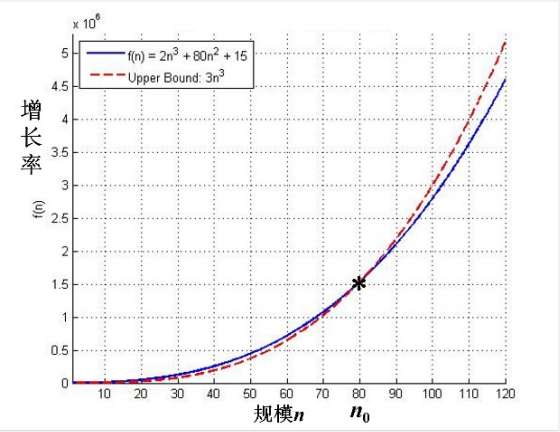


图1.8　大O表示法的渐进曲线示例

被称为问题的基本操作的**原操作**应是其重复执行次数和算法的执行时间成正比的原操作，多数情况下它是最深层循环内的语句中的原操作，它的执行次数和包含它的语句的频度相同。

语句的**频度**（frequency count）指的是该语句重复执行的次数，例如，在下列3个程序段中：

```
(a) { ++x;    s=0; }
(b) for (i=1; i<=n; ++i) { ++x;    s+=x; }
(c) for (j=1; j<=n; ++j)
        for(k=1; k<=n; ++k){ ++x;    s+=x; }
```

含基本操作“++x;”的语句的频度分别为1、n和n2，
这3个程序段的时间复杂度分别为0(1)、0(n)和0(n2)，
分别称为**常数阶**、**线性阶**和**二次阶**。

算法还可能呈现的时间复杂度有**对数阶**0(1ogn)、**指数阶**0(2n)等。不同数量级时间复杂度的性状如图1.9所示。

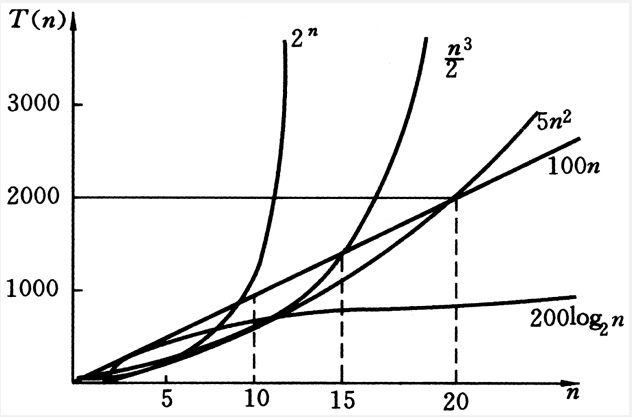


图1.9　常见函数的增长率

以下是由低到高排列的增长率：

1, loglog$n$, $\sqrt{\log n}$, $\sqrt{n}$, $\log^2 n$, $2^{\log n}$, n, $\log(n!)$, $n\log n$, n^2, 2^n, $n!$, 2^{2^n}

一般情况下，对一个问题（或一类算法）只需选择一种基本操作来讨论算法的时间复杂度即可，有时也需要同时考虑几种基本操作，甚至可以对不同的操作赋予不同权值，以反映执行不同操作所需的相对时间，这种做法便于综合比较解决同一问题的两种完全不同的算法。

对于程序的基本结构，有以下**计算基本操作频度的法则**

1. **循环语句**。一个循环语句中的基本操作的频度是其迭代次数，也就是循环次数的常数倍。
2. **嵌套循环语句**。最内层的基本操作的频度是各层循环的次数的乘积的常数倍。
3. **顺序语句**。一组顺序语句的基本操作频度是每个语句（的基本操作）频度之和。这些语句可以是循环语句或条件语句。
4. **条件语句**。常见的条件语句是**if/else**或**switch/case**语句，也称为分支语句。条件语句的基本操作频度是条件表达式的频度+各分支的频度最大值。

算法中基本操作重复执行的次数通常随问题的输入数据集而异。例如在起泡排序的算法中：

```
void bubble_sort(int a[], int n) { // 将数组a中整数序列重排成自小至大有序序列
 Status change;
 for (int i=n-1, change=TRUE; i>=1 && change; --i) {
  change = FALSE;
  for (int j=0; j<i; ++j)
   if (a[j]>a[j+1]) { swap(&a[j], &a[j+1]);  change=TRUE; } //交换函数swap
 }
}
```

“交换序列中相邻两个整数”为基本操作。

当数组a中初始序列已经从小到大有序，基本操作的执行次数为0，这属于“最好情况”；

当初始序列为从大到小有序时，基本操作的执行次数为$n(n\text{-}1)/2$，是“最坏情况”。

对这类算法的分析，一种解决办法是计算平均值，即考虑它对所有可能的输入数据集的期望值，此时的时间复杂度为算法的**平均时间复杂度**。

如假设a中初始数据可能出现$n!$种的排列情况的概率相等，则起泡排序的**平均时间复杂度**$T_{avg}(n) = \boldsymbol{O}(n^2)$。

在很多情况下，各种输入数据集出现的概率难以确定，算法的平均时间复杂度也就难以确定。

更可行也更常用的办法是讨论算法在最坏情况下的时间复杂度，即分析最坏情况以估算算法执行时间的一个上界（与此相对，最好情况的执行时间是下界）。

例如，上述起泡排序的最坏情况为a中初始序列为**从大到小的逆序**，则该算法在最坏情况下的时间复杂度为$T(n)=\boldsymbol{O}(n^2)$。

在以后讨论的时间复杂度，除特别指明外，均指**最坏情况时间复杂度**，简称**最坏时间复杂度**。

很多时候，一个算法的最坏情况出现的次数极少，比率仅为c/n，c为小常数。

仅用最坏时间复杂度可能高估算法的时间耗费，尤其对大规模数据集的算法。

可采用**平摊分析**（Amortied analysis）来估算算法时间复杂度。平摊也称为**均摊**、**摊分**或**摊还**。与平均时间复杂度不同，平摊分析不使用概率，是对含最坏情况的一组连续的操作作具体分析。

平摊策略有多种，较简明的是聚集分析：若每n个$\boldsymbol{O}(1)$的操作，才发生1个$\boldsymbol{O}(n)$的操作，则将最坏情况的$\boldsymbol{O}(n)$摊分到其他n个$\boldsymbol{O}(1)$操作后，这n+1个操作的**平摊时间复杂度**是$\boldsymbol{O}(1)$。



1.4.4 算法的存储空间需求

类似于算法的时间复杂度，以**空间复杂度**（space complexity）作为算法所需存储空间的量度，记作

$$S(n)=O(f(n)) \tag{1-6}$$

其中n为问题的规模（或大小）。

一个上机执行的程序除了需要存储空间来寄存本身所用指令、常数、变量和输人数据外，也需要一些对数据进行操作的工作单元和存储一些为实现计算所需信息的辅助空间。

若输入数据所占空间只取决于问题本身，和算法无关，则只需要分析除输入和程序之外的额外空间，否则应同时考虑输入本身所需空间（和输入数据的表示形式有关）。

若额外空间相对于输入数据量来说是常数，则称此算法为**原地工作**，第10章讨论的有些排序算法就属于这类。

如果所占空间量依赖于特定的输入，则除特别指明外，均按最坏情况分析。